



UNIVERSIDADE DE
VASSOURAS

ENGENHARIA DE SOFTWARE



UNIVERSIDADE DE VASSOURAS

Curso de Graduação em Engenharia de Software

Prof. Márcio Garrido

Mestre em Engenharia Elétrica e Telecomunicações

Especialista em Segurança da Informação

Especialista em Engenharia Elétrica e Automação

MBA Engenharia Segurança do Trabalho

Bacharel em Engenharia de Software

Bacharel em Sistema da Informação e Analista de

Sistemas



E-mail:

marcio.garrido@univassouras.edu.br



Disciplina: Laboratório de Programação Full Stack



Agenda da Aula

Disciplina: Laboratório de Programação Full Stack

Aula 5 — Django na Prática II: as telas do sistema

1. O ciclo



URL → View → Template

2. Views



a lógica de cada página

3. URLs



conectar endereços a views

4. Templates



montar o HTML com dados

5. Herança de templates



layout sem repetição

6. Forms



receber dados do usuário

Onde Paramos

Aula 4 em uma frase



Já temos dados no banco

Na Aula 4 criamos o projeto, o app e o model Livro, aplicamos as migrações e cadastramos livros pelo Django Admin. Os dados existem no PostgreSQL — mas o usuário final ainda não vê nada.



Hoje: as telas

Vamos exibir esses livros em páginas de verdade, montadas com views e templates.



E os formulários

No fim, deixaremos o usuário cadastrar livros pela própria página — sem depender do Admin.

Antes de Começar: Uma Pergunta

Por que o `urls.py` não vem pronto como os outros arquivos?

Na Aula 4, o startapp criou automaticamente `models.py`, `views.py`, `admin.py`... mas não o `urls.py`. Isso não é uma falha do Django — é uma decisão de design bem intencional.

A analogia do Lego

Pense no app como uma peça de Lego: ela já vem pronta, com sua forma e função definidas — isso é o `models.py` e o `views.py`. Mas ONDE essa peça se encaixa na maquete final é decisão de quem está montando o projeto, não do fabricante da peça.

Por isso o "encaixe" — o `urls.py` — não vem pronto: é você, como integrador do projeto, quem decide e declara isso.

A ideia central: `models.py` e `views.py` descrevem o que o app É e FAZ. O `urls.py` descreve ONDE ele fica exposto no site — e isso muda de projeto para projeto.

Quatro Razões Bem Concretas

Não é só filosofia — isso resolve problemas reais

Apps são reutilizáveis



Um app de terceiros (ex.: sistema de login pronto) não tem como saber, quando foi escrito, se você vai montá-lo em `/login/`, `/contas/` ou `/auth/`.

O mesmo app, em vários lugares



O Django permite incluir o mesmo app duas vezes, em prefixos diferentes — um app de "comentários" em `/blog/` e também em `/produtos/`, por exemplo.

Nem todo app precisa de URL



Apps que só guardam models compartilhados ou só aparecem no Admin nunca são acessados por uma URL própria — um `urls.py` automático geraria arquivo morto.

Transparência do roteamento



"Explícito é melhor que implícito": o `urls.py` do projeto é onde se vê o mapa completo do site. Se cada app se registrasse sozinho, essa visão central se perderia.

É por isso que, na Aula 4 e nesta aula, você sempre cria o `urls.py` do app à mão — e o `urls.py` do projeto "inclui" cada um deles, com o prefixo que VOCÊ escolhe.

Relembrando

O Ciclo URL → View → Template

É o caminho que toda página Django percorre. Hoje vamos construir cada uma dessas três peças:



1. URL

o endereço que o usuário acessa



2. View

a função que busca os dados e decide o que mostrar



3. Template

o HTML que exibe os dados na tela



A ordem importa

A URL chama a View, que usa o Model (dados) e escolhe o Template. Vamos montar na mesma ordem.

Passo 1

Views — a Lógica da Página

A view decide o que a página vai mostrar

É uma função Python que recebe a requisição, busca o que precisa e devolve uma resposta — normalmente, uma página.



O papel da view

Pense na view como o "cérebro" da página: ela pega os dados (via ORM), aplica qualquer regra necessária e entrega tudo pronto para o template exibir. Toda página tem uma view por trás.

Uma View que Lista os Livros

acervo/views.py

acervo/views.py

```
from django.shortcuts import render
from .models import Livro

def lista_livros(request):
    livros = Livro.objects.all()    # busca no banco
    return render(
        request, 'acervo/lista.html',
        {'livros': livros}    # envia ao template
    )
```



A função render

render junta três coisas: a requisição, o nome do template e um "contexto" (os dados). É o atalho mais usado do Django — busca os dados com o ORM e entrega ao template em uma linha.

Passo 2

URLs — Conectando Endereços

A URL diz qual view responde cada endereço

Sem o mapeamento de URLs, o Django não sabe qual função chamar quando alguém acessa uma página.

Duas camadas de URLs — uma delas você cria



O projeto já vem com um `urls.py` principal (gerado pelo startproject). Mas o startapp NÃO cria um `urls.py` dentro do app — esse arquivo você cria manualmente. O projeto então "inclui" o `urls.py` de cada app, para organizar as rotas por app em vez de um arquivo gigante.

Configurando as URLs

Duas etapas: no app e no projeto

acervo/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path(
        'livros/',
        views.lista_livros,
        name='lista'
    ),
]
```

biblioteca/urls.py

```
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path(
        '', include('acervo.urls')
    ),
]
```



O parâmetro name

Dar um name à rota (aqui, 'lista') permite referenciá-la pelo apelido nos templates e views, em vez de escrever o endereço na mão. Se a URL mudar, nada quebra.



Este arquivo não existe por padrão

startapp não cria acervo/urls.py. Crie esse arquivo você mesmo dentro da pasta do app (com o editor de código ou touch acervo/urls.py) antes de escrever o código ao lado.

Passo 3

Templates — a Tela

O template transforma dados em HTML

É um arquivo HTML com "espaços" que o Django preenche com os dados vindos da view.



A Django Template Language (DTL)

Além do HTML normal, os templates entendem uma linguagem simples: `{{ }}` para mostrar valores e `{% %}` para lógica (laços e condições). Nada de programação pesada — só o suficiente para exibir dados.

A Sintaxe dos Templates

Dois símbolos que você vai usar o tempo todo



{{ variavel }}

Mostra um valor. Ex.: {{ livro.titulo }} escreve o título do livro na página.



{% tag %}

Executa lógica. Ex.: {% for %, {% if %, {% block %, {% extends %}.

Tag	Para que serve
{% for x in lista %}	repete um bloco para cada item
{% if condicao %}	mostra algo só quando a condição é verdadeira
{% extends %} / {% block %}	herança de template (layout base)

O Template da Lista

acervo/templates/acervo/lista.html

lista.html

```
<h1>Acervo</h1>
<ul>
  {% for livro in livros %}
    <li>{{ livro.titulo }} – {{ livro.autor }}</li>
  {% empty %}
    <li>Nenhum livro cadastrado.</li>
  {% endfor %}
</ul>
```



HTML + dados

O `{% for %}` percorre a lista que a view mandou; o `{{ }}` mostra cada campo. O `{% empty %}` cobre a lista vazia.

O Fluxo Completo de uma Página

Juntando as três peças

1

Usuário acessa `/livros/`

a URL identifica a rota

2

`urls.py` chama a view

`lista_livros(request)`

3

View busca no banco

`Livro.objects.all()` via ORM

4

`render()` usa o template

injeta os livros no `lista.html`

5

HTML volta pronto

o usuário vê a lista na tela



Este é o padrão de toda página

Toda tela do seu projeto vai seguir exatamente estes cinco passos. Domine este fluxo e você domina o Django.

Passo 4

Herança de Templates

Escreva o layout uma vez, use em todas as páginas

Cabeçalho, menu e rodapé se repetem em todo site. Em vez de copiar, o Django deixa você herdar de um template-base.



DRY na prática

Um arquivo base.html define a estrutura comum e deixa "buracos" (blocks). Cada página preenche só o seu miolo. Mudou o menu? Muda em um lugar só, e todas as páginas acompanham.

Herança na Prática

base.html e a página que o estende

base.html

```
<html><body>
  <header>Biblioteca</header>

  {% block conteudo %}
  {% endblock %}

  <footer>© 2026</footer>
</body></html>
```

lista.html

```
{% extends 'base.html' %}

{% block conteudo %}
  <h1>Acervo</h1>
  <!-- lista aqui -->
{% endblock %}
```



Como funciona

A página usa {% extends %} para herdar o base e preenche o {% block %} com seu conteúdo. Header e footer vêm de graça.

Arquivos Estáticos: CSS e Imagens

Deixando o site com cara de site

template com CSS

```
{% load static %}  
<link rel="stylesheet" href="{% static 'acervo/estilo.css' %}">
```



A pasta static/

Coloque CSS, imagens e JS numa pasta static/ dentro do app. O Django serve esses arquivos.



A tag {% static %}

Gera o caminho certo para cada arquivo — nunca escreva o caminho na mão.

Passo 5

Forms — Recebendo Dados

Deixando o usuário cadastrar pela página

Até agora só exibimos dados. Com forms, o usuário envia dados — e o Django valida tudo para você.



Por que usar os forms do Django

Em vez de montar o HTML do formulário e validar campo a campo na mão, o Django gera o formulário a partir do model e cuida da validação, das mensagens de erro e da segurança. Menos código, menos bug.

O ModelForm

Um formulário gerado a partir do model

acervo/forms.py

```
from django import forms
from .models import Livro

class LivroForm(forms.ModelForm):
    class Meta:
        model = Livro
        fields = ['titulo', 'autor', 'ano']
```



De graça a partir do model

O ModelForm olha para o model Livro e cria os campos automaticamente. Você só diz quais campos quer no formulário — os tipos, os rótulos e a validação vêm prontos.

A View que Processa o Formulário

GET mostra, POST salva

acervo/views.py

```
def novo_livro(request):  
    if request.method == 'POST':  
        form = LivroForm(request.POST)  
        if form.is_valid():  
            form.save() # grava no banco  
            return redirect('lista')  
    else:  
        form = LivroForm()  
    return render(request, 'acervo/form.html', {'form': form})
```



Dois caminhos numa view só

GET (abrir a página) mostra o formulário vazio; POST (enviar) valida e salva. `is_valid()` faz a checagem.

O Template do Formulário

Simple assim

form.html

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button>Salvar</button>
</form>
```



{% csrf_token %}

Proteção obrigatória contra ataques CSRF. O Django recusa o envio sem ela — sempre inclua.



{{ form.as_p }}

Renderiza todos os campos do formulário automaticamente, cada um em um parágrafo.

Validação Automática

O Django cuida dos erros para você



is_valid()

Confere se todos os campos estão corretos antes de salvar. Se algo falhar, não grava nada.



Mensagens de erro

Se um campo obrigatório vier vazio ou com tipo errado, o Django mostra a mensagem ao lado do campo.



Segurança inclusa

Proteção contra CSRF e limpeza dos dados vêm automáticas — menos brechas no seu sistema.



Regras próprias

Precisa de uma regra específica? Dá para adicionar validações personalizadas no form.

O CRUD Está Quase Completo

O que já temos e o que falta

Operação	Como fazemos	Status
Create (criar)	formulário + form.save()	✓ visto hoje
Read (ler)	view + template de lista	✓ visto hoje
Update (editar)	mesmo form, com instance	→ Aula 6
Delete (apagar)	view de exclusão	→ Aula 6



Falta pouco

Já sabemos criar e listar. Na Aula 6 fechamos o CRUD com editar e apagar — e você terá um app completo.

Boas Práticas de Views e Templates

Hábitos que mantêm o código limpo



Lógica na view

Busca e regras ficam na view; o template só exibe. Nunca coloque cálculo no HTML.



Use herança

Um base.html para o layout comum. Não copie cabeçalho e rodapé em cada página.



URLs com name

Refira as rotas pelo apelido. Facilita manutenção e evita links quebrados.



ModelForm sempre

Para formulários ligados a models, o ModelForm poupa muito código e erros.

Erros Comuns Nesta Etapa

E como evitá-los



Template não encontrado

Caminho errado da pasta templates/ ou nome do arquivo diferente do render.



Esquecer o csrf_token

Formulário POST sem {% csrf_token %} — o Django bloqueia o envio.



Não incluir as urls

Criar acervo/urls.py e esquecer o include no urls.py do projeto.



Lógica no template

Tentar fazer contas ou regras no HTML em vez de na view.

Glossário da Aula

Os termos que você leva desta aula

Termo	Significado em uma frase
View	Função Python que busca dados e devolve uma resposta
render()	Atalho que junta requisição, template e dados numa página
DTL	Django Template Language: {{ }} para valores, {% %} para lógica
ModelForm	Formulário gerado automaticamente a partir de um model
CSRF token	Proteção que valida que o envio veio do seu próprio site

Resumo da Aula 5

O que você construiu hoje



Views & URLs

Funções que buscam dados e rotas que as conectam aos endereços.



Templates

HTML com DTL e herança — exibindo os dados do banco na tela.



Forms

ModelForm para o usuário cadastrar, com validação automática.



A conclusão da aula

Agora o sistema tem telas de verdade: o usuário vê a lista de livros e cadastra novos pela própria página. Fechamos o ciclo URL → View → Template. Na Aula 6, completamos o CRUD e você escolhe seu projeto.

Onde Estamos no Cronograma

Reta final dos fundamentos

Aula	Tema	Status
3	Back-end em perspectiva: Python e Django	✓ concluída
4	Django I: projeto, models, ORM e Admin	✓ concluída
5	Django II: views, URLs, templates e forms	● hoje
6	CRUD completo + catálogo de projetos e escolha	→ próxima



A última aula de fundamentos

Na Aula 6 fechamos o CRUD e apresentamos os 10 projetos. A partir da Aula 7, é laboratório: você constrói o seu.

Para a Próxima Aula

Aula 6 — CRUD Completo e Escolha do Projeto



Próxima aula: fechando o CRUD

Vamos completar o app com editar e apagar livros, revisar tudo o que aprendemos — e apresentar a lista de 10 projetos para você escolher o que vai construir no laboratório.



Comece a pensar

Qual dos temas mais combina com você? Um sistema de biblioteca, uma agenda, um controle financeiro? A escolha é sua.



Combinado

Traga o projeto de hoje funcionando (listar e cadastrar). Vamos evoluí-lo junto na próxima aula.

Dica de Estudo

Para fixar antes da próxima aula



Refaça o fluxo sozinho

Sem olhar o material, tente criar uma nova página do zero: uma view, sua URL e um template. Repetir o ciclo URL → View → Template é o que fixa o aprendizado.



Documentação oficial

A documentação do Django é excelente e cheia de exemplos. O tutorial oficial ("Escrevendo seu primeiro app") cobre exatamente o que vimos.



Você já consegue muito

Com views, templates e forms, você já é capaz de construir a maior parte de qualquer sistema web.

Dúvidas?

Aula 5 — Django na Prática II

Obrigado!

Dados no banco viram páginas na tela — esse é o coração do Django.



Contato

marcio.garrido@univassouras.edu.br